

개념

▼ 1.1

인공지능 > 머신러닝 > 딥러닝

머신러닝과 딥러닝의 차이

- 머신러닝과 딥러닝 모두 학습 모델을 제공하여 데이터를 분류할 수 있는 기술

머신러닝은 주어진 데이터를 인간이 먼저 처리(전처리)함

머신러닝은 범용적인 목적으로 제작된 것으로 데이터의 특징을 스스로 추출하지 못함

→ 데이터의 특징을 컴퓨터가 인식하고 학습할 수 있도록 인간이 처리해 주어야 함

- 딥러닝은 인간이 하던 작업을 생략함

→ 대량의 데이터를 신경망에 적용하면 컴퓨터가 스스로 분석한 후 답을 찾음

▼ 1.2

머신 러닝이란

머신 러닝은 인공 지능의 한 분야로, 컴퓨터 스스로 대용량 데이터에서 지식이나 패턴을 찾아 학습하고 예측을 수행하는 것

즉, 컴퓨터가 학습할 수 있게 하는 알고리즘과 기술을 개발하는 분야

머신 러닝 학습 과정

머신 러닝은 크게 **학습 단계**와 **예측 단계**로 구분할 수 있음

- 학습 단계

훈련 데이터를 머신러닝 알고리즘에 적용하여 학습시킴

→ 학습 결과로 모델이 생성됨

- 예측 단계

학습 단계에서 생성된 모델에 새로운 데이터를 적용하여 결과를 예측함

특성 추출 : 데이터별로 어떤 특징을 가지고 있는지 찾아내고, 그것을 토대로 데이터를 벡터로 변환하는 작업

머신 러닝의 주요 구성 요소

머신 러닝의 주요 구성 요소는 데이터와 모델(모형)임

- 데이터

데이터는 머신 러닝이 학습 모델을 만드는 데 사용. 훈련 데이터가 나쁘다면 실제 현상의 특성을 제대로 반영할 수 없으므로, 실제 데이터의 특징이 잘 반영되고 편향되지 않는 훈련 데이터를 확보하는 것이 중요. 학습에 필요한 데이터가 수집되었다면 훈련 데이터셋과 테스트 데이터셋 용도로 분리해서 사용

보통 데이터의 80%는 훈련용, 20%는 테스트용으로 분리해서 사용함

- 모델

모델은 머신 러닝의 학습 단계에서 얻은 최종 결과물로 가설이라고도 함. 예를 들어 입력 데이터의 패턴은 A와 같다는 가정을 머신 러닝에서는 모델이라고 함

모델의 학습 절차:

1. 모델(또는 가설) 선택
2. 모델 학습 및 평가
3. 평가를 바탕으로 모델 업데이트

→ 이 세 단계를 반복하면서 주어진 문제를 가장 잘 풀 수 있는 모델을 찾음

훈련 데이터셋, 검증 데이터셋, 테스트 데이터셋

전체 데이터셋은 크게 훈련 데이터셋과 테스트 데이터셋으로 분리하여 사용. 훈련 데이터셋을 다시 훈련 데이터셋과 검증 데이터셋으로 분리해서 사용하기도 함

검증 데이터셋

검증 데이터셋을 사용하는 이유는 모델을 평가하기 위해서. 훈련 데이터셋으로 모델을 학습시킨 후 모델이 잘 예측하는지 그 성능을 평가하기 위해 사용. 하지만 검증 데이터셋은 훈련 데이터셋의 일부를 떼어서 사용하기 때문에, 학습에 사용되는 데이터셋의 양이 많지 않다면 검증 데이터셋을 사용하는 것은 좋지 않음

검증 데이터셋을 사용하는 이유

1. 테스트 데이터셋에 대한 성능을 미리 확인할 수 있음
 - 새롭게 수집될 데이터에 대해 정확한 예측을 하는지 평가할 수 있음
2. 모델의 성능을 높이는 데 도움을 줌
 - 훈련 데이터셋의 정확도는 높는데 검증 데이터셋의 정확도가 낮다면 훈련 데이터셋에 과적합이 일어났을 가능성이 있음
 - 이 경우 정규화를 하거나 에포크를 줄이는 방식으로 과적합을 막을 수 있음

머신 러닝 학습 알고리즘

- 지도 학습

정답이 무엇인지 컴퓨터에 알려 주고 학습시키는 방법

- 비지도 학습

정답을 알려 주지 않고, 유사한 데이터를 클러스터링(범주화)하여 예측하는 학습 방법

→ 데이터 간 거리 측정을 통해 특징이 유사한 데이터끼리 클러스터링으로 묶어서 분류함

- 강화 학습

자신의 행동에 대한 보상을 받으며 학습을 진행. 에이전트가 변화하는 환경에 따라 다른 행동을 취하고, 행동에 따라 보상을 얻음

→ 보상이 커지는 행동은 자주 하도록 하고, 줄어드는 행동은 덜 하도록 하여 학습을 진행

▼ 1.3

1) 딥러닝 학습 과정

- 데이터준비

파이토치 (<https://tutorials.pytorch.kr/>), 케라스(<https://keras.io>), 캐글

- 모형 정의

은닉층 개수에 따른 과적합과 성능의 상충 관계를 고려해 신경망 생성

- 모델 컴파일

활성화 함수, 손실 함수, 옵티마이저 선택

연속형 : 평균 제곱 오차

이진 분류 : 크로스 엔트로피

- 모델 훈련

전체 훈련 데이터셋에서 일정한 묶음으로 나누어 처리할 수 있는 배치와 훈련의 횟수인 에포크 선택

최적의 파라미터와 하이퍼 파라미터 찾음

- 모델 예측

검증 데이터셋을 생성한 모델에 적용하여 실제로 예측을 진행

심층 신경망 : 데이터셋의 어떤 특성들이 중요한지 스스로에게 가르쳐 줄 수 있는 기능이 있음

역전파 : 가중치 값 업데이트에 중요

2) 딥러닝 학습 알고리즘

1. 지도 학습

합성곱 신경망 (CNN) : 이미지 분류, 이미지 인식, 이미지 분할

순환 신경망 (RNN) : 시계열 데이터에 사용

→ 역전파 과정에서 기울기 소멸 문제가 발생하는 단점

→ LSTM : 이를 개선하고자 망각, 입력, 출력 게이트를 도입

2. 비지도 학습

- 워드 임베딩 : 단어를 벡터로 표현하는 기술

- 군집 : 아무런 정보가 없는 상태에서 데이터를 분류하는 방법

한 클러스터 안의 데이터는 매우 비슷하게 구성하고 다른 클러스터의 데이터와 구분 되도록 나누는 것이 목표

3. 전이 학습

: 사전에 학습이 완료된 모델을 가지고 우리가 원하는 학습에 미세 조정 기법을 이용하여 학습시키는 방법

→ 사전 학습이 완료된 모델이 필요하고 이를 어떻게 활용하는지에 대한 접근 방법이 필요

사전 학습 모델은 풀고자 하는 문제와 비슷하면서 많은 데이터로 이미 학습이 되어 있는 모델

사전 학습 모델을 활용하는 방법 : 특성 추출 / 미세 조정 기법

▼ 2.1

1. 특징 및 장점

특징 : GPU에서 텐서 조작 및 동적 신경망 구축이 가능한 프레임워크

GPU : 연산 속도 빠르게 하는 역할

텐서 : 파이토치의 데이터 형태 / 단일 데이터 형식으로 된 자료들의 다차원 행렬

동적 신경망 : 훈련을 반복할 때마다 네트워크 변경이 가능한 신경망

ex) 학습 중에 은닉층을 추가하거나 제거하는 등 모델의 네트워크 조작이 가능

벡터 : 1차원 배열 형태

행렬 : 행과 열로 표현되는 2차원 배열 형태

텐서 : 3차원 이상의 배열 형태

신경망은 연산 그래프를 이용하여 계산을 수행

네트워크가 학습될 때 손실 함수의 기울기가 가중치와 바이어스를 기반으로 계산되며, 이후 경사하강법을 사용하여 가중치가 업데이트 됨. 이때 연산 그래프를 이용하여 이 과정이 효과적으로 수행됨

파이토치 장점 : 단순함 (효율적인 계산) / 성능 (낮은 CPU 활용) / 직관적인 인터페이스

2. 파이토치의 아키텍처

첫 번째 계층 : 파이토치 API - 텐서에 대한 처리 / 신경망 구축과 훈련 도움

두 번째 계층 : 파이토치 엔진 - 다차원 텐서 및 자동 미분 처리

세 번째 계층 : 텐서에 대한 연산 처리. 실질적 계산을 위한 라이브러리가 위치함

텐서는 1차원이든 N차원이든 메모리에 저장할 때는 1차원 배열 형태가 됨

스토리지 : 변환된 1차원 배열

오프셋 : 텐서에서 첫 번째 요소가 스토리지에 저장된 인덱스

스트라이드 : 각 차원에 따라 다음 요소를 얻기 위해 건너뛰기가 필요한 스토리지의 요소 개수

= 메모리에서의 텐서 레이아웃을 표현

오프셋과 스트라이드는 데이터 자체가 아닌 행렬 / 텐서를 구분하기 위해 사용

▼ 2.2

1. 텐서 다루기

2. 데이터 준비

데이터가 이미지일 경우 : 분산된 파일에서 데이터를 읽은 후 전처리를 하고 배치 단위로 분할하여 처리

데이터가 텍스트일 경우 : 임베딩 과정을 거쳐 서로 다른 길이의 시퀀스를 배치 단위로 분할하여 처리

- 단순히 파일을 불러와서 사용

판다스 라이브러리를 이용하여 파일을 불러옴

- 커스텀 데이터셋을 만들어 사용

데이터를 한 번에 메모리에 불러와서 훈련시키면 시간과 비용 측면에서 효율적이지 않으므로 조금씩 나누어 불러서 사용하는 방식

- 파이토치에서 제공하는 데이터셋 사용

3. 모델 정의

모델을 정의하기 위해 모듈을 상속한 클래스를 사용

- 계층 : 모듈 또는 모듈을 구성하는 한 개의 계층으로 합성곱층, 선형계층 등이 있음
- 모듈 : 한 개 이상의 계층이 모여서 구성된 것으로, 모듈이 모여 새로운 모듈을 만들 수 있음
- 모델 : 최종적으로 원하는 네트워크로, 한 개의 모듈이 모델이 될 수도 있음

- 단순 신경망을 정의하는 방법
- `nn.Module()`을 상속하여 정의하는 방법

파이토치에서 `nn.Module` 을 상속받는 모델은 기본적으로 `__init__()` 과 `forward()` 함수를 포함. `__init__()` 에서는 모델에서 사용될 모듈(`nn.Linear`, `nn.Conv2d`), 활성화 함수 등을 정의하고, `forward()` 함수에서는 모델에서 실행되어야 하는 연산을 정의

- `Sequential` 신경망을 정의하는 방법

`nn.Sequential` 을 사용하면 `__init__()` 에서 사용할 네트워크 모델들을 정의해 줄 뿐만 아니라 `forward()` 함수에서는 모델에서 실행되어야 할 계산을 좀 더 가독성이 뛰어나게 코드로 작성할 수 있음. 또한, `Sequential` 객체는 그 안에 포함된 각 모듈을 순차적으로 실행해 주는데 다음과 같이 코드를 작성할 수 있음.

- 함수로 신경망을 정의하는 방법

`Sequential`을 이용하는 것과 동일하지만, 함수로 선언할 경우 변수에 저장해 놓은 계층들을 재사용할 수 있는 장점이 있음. 하지만 모델이 복잡해지는 단점도 있음. 참고로 복잡한 모델의 경우에는 함수를 이용하는 것보다는 `nn.Module()` 을 상속받아 사용하는 것이 편리

4. 모델의 파라미터 정의

손실 함수(loss function):

: 학습하는 동안 출력과 실제 값 사이의 오차를 측정. 즉, 예측값과 실제 값의 오차를 구해서 모델의 정확성을 측정

- BCELoss: 이진 분류를 위해 사용
- CrossEntropyLoss: 다중 클래스 분류를 위해 사용
- MSELoss: 회귀 모델에서 사용

옵티마이저(optimizer)

: 데이터와 손실 함수를 바탕으로 모델의 업데이트 방법을 결정

- optimizer는 `step()` 메서드를 통해 전달받은 파라미터를 업데이트
- 모델의 파라미터별로 다른 기준(예: 학습률)을 적용시킬 수 있습니다.

학습률 스케줄러

미리 지정한 횟수의 에포크를 지날 때마다 학습률 감소시켜줌. 학습 초기에는 빠른 학습을 진행하다가 전역 최소점 근처에 다다르면 학습률을 줄여 최적점을 찾아갈 수 있도록 함

지표

: 훈련과 테스트 단계를 모니터링

5. 모델 훈련

파이토치는 기울기 값을 계산하기 위해 `loss.backward()` 메서드를 이용하는데, 이것을 사용하면 새로운 기울기 값이 이전 기울기 값에 누적하여 계산됨. RNN 모델에는 효과적이지만 누적 계산이 필요하지 않는 모델에 대해서는 불필요.

→ 입력 값을 모델에 적용하기 전에 `optimizer.zero_grad()` 메서드를 이용하여 기울기 = 미분값을 초기화

다음은 `loss.backward()` 메서드를 이용하여 기울기를 자동 계산. `loss.backward()` 는 배치 반복될 때마다 오차가 중첩적으로 쌓이게 되므로 매번 `zero_grad()` 를 사용하여 미분 값을 0으로 초기화.

6. 모델 평가

함수나 모듈을 이용해 모델을 평가

사이킷런의 metrics 모듈에서 제공하는 `confusion_matrix`, `accuracy_score` 와 `classification_report` 클래스를 이용하면 쉽게 정확도를 찾을 수 있음

7. 훈련 과정 모니터링

텐서보드를 이용하면 학습에 사용되는 각종 파라미터 값이 어떻게 변화하는지 손쉽게 시각화하여 살펴볼 수 있으며 성능을 추적하거나 평가하는 용도로도 사용할 수 있음

설정 → 기록 → 사용해 모델 구조 살펴봄

▼ 2.3

▼ 2.4